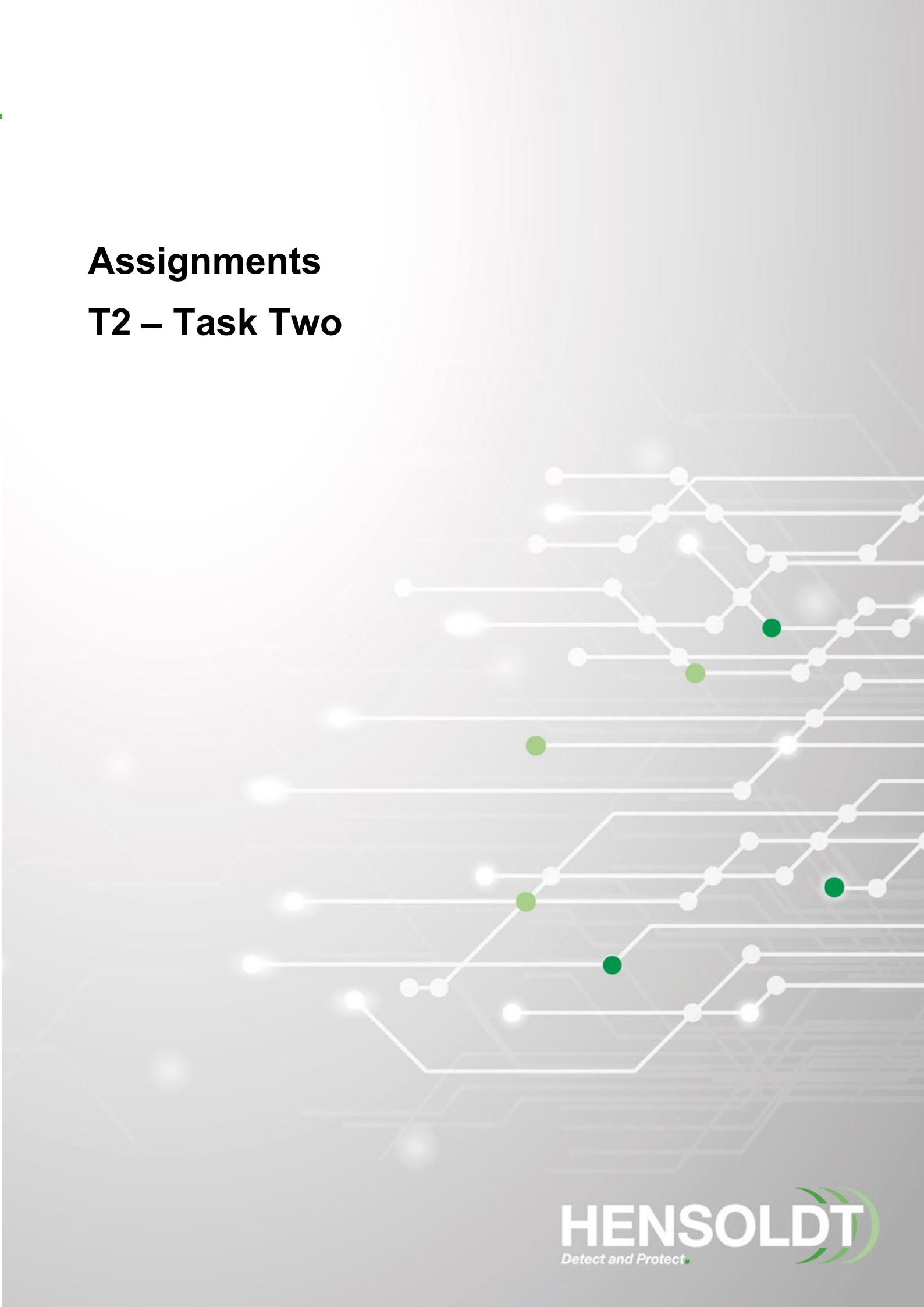
Assignments

T2 – Task Two



HENSOLDT
Detect and Protect.

Notice

The information in this document is subject to change without notice, and should not be construed as a commitment by HENSOLDT Cyber.

HENSOLDT Cyber provides no warranty, express or implied, including warranties of merchantability and fitness for a particular purpose for the information contained in this document, and assumes no responsibility for any errors that may appear in this document. In no event shall HENSOLDT Cyber or any of its suppliers be liable for direct, indirect, special, incidental or consequential damages of any nature or kind arising from the use of this document, or from the use of any hardware or software described in this document, even if HENSOLDT Cyber or its suppliers have been advised of the possibility of such damages.

Trademarks

TRENTOS® is a trademark of HENSOLDT Cyber GmbH.

sel4® is a trademark of LF Projects, LLC.

Table of Contents

The main task of this assignment is to replace the existing `RamDisk` based storage solution by a real hardware based storage solution on the RPi 3 B+. Therefore, we will utilize the SPI NOR flash memory breakout board, which you can find in the TRENTOS Development Kit. More information regarding the SPI NOR flash memory and the respective driver can be found in the TRENTOS-M Handbook, chapter "TRENTOS-M RPi_SPI_Flash". The figure below describes the modified setup we want to create:

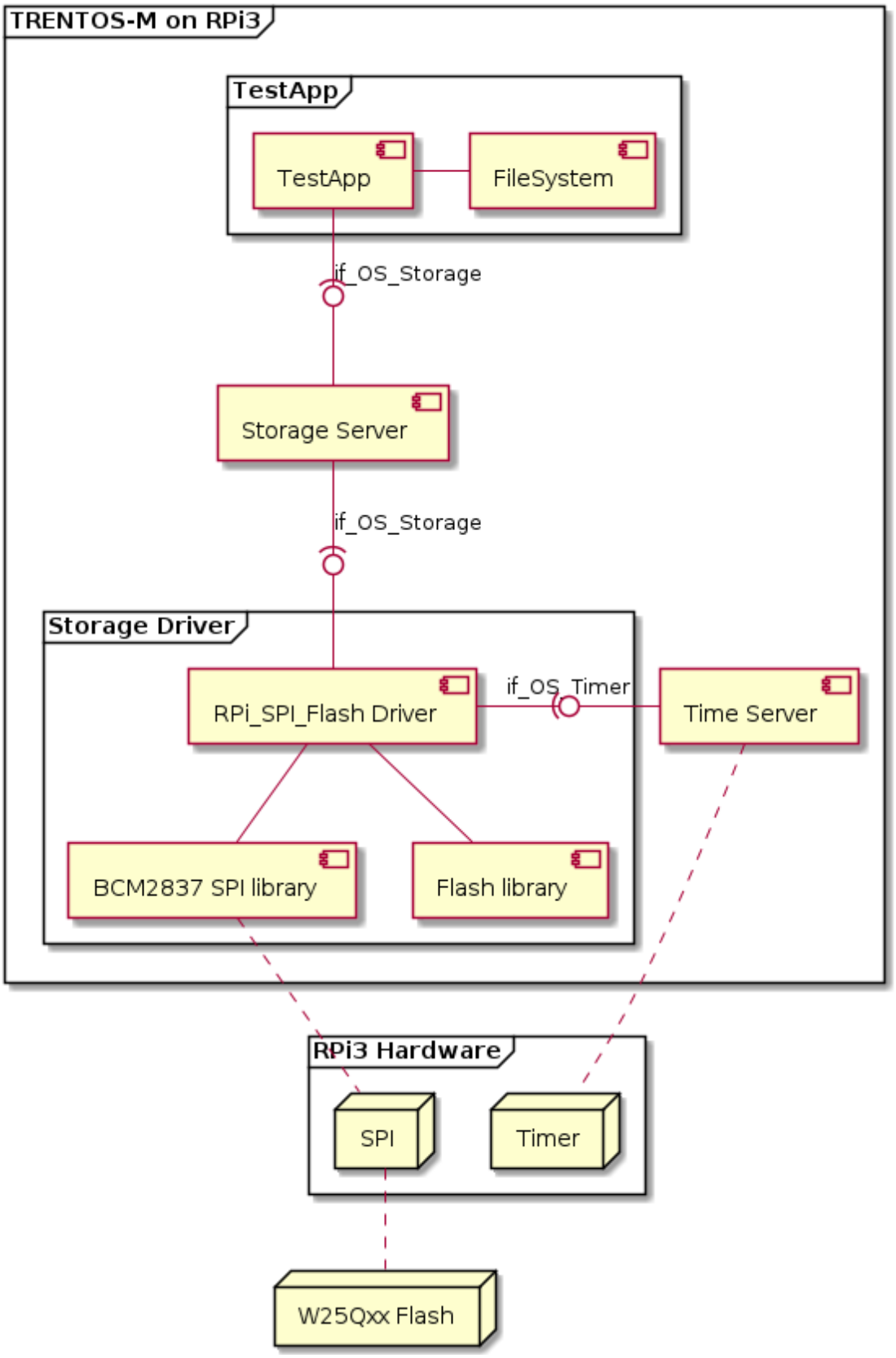


Figure 1: RPi_SPI_Flash Driver - Architecture

1 Learning Goals

As a developer you should understand

- the usage of the TRENTOS-M standard interface `if_OS_Storage`
- the concept of simply exchanging underlying storage backends (e.g. the provided RamDisk) by relying on the abstraction provided by the TRENTOS-M Storage API
- the differences in connecting virtual and physical hardware devices to TRENTOS-M

2 What to do

In order to make use of the flash memory, we have to connect two hardware devices to our setup:

- the actual NOR flash memory which utilizes the SPI bus
- a RPi internal timer resource (as the `RPi_SPI_Flash` requires a timer)

For both hardware devices (flash memory, timer) as well as the SPI bus, TRENTOS-M already provides the respective drivers. At this point, there is no need for us to create drivers on our own, we are free to re-use the provided resources. But as you can easily see, we have to adapt our current setup in a certain way to reflect the changes depicted in the figure above:

- we re-use the TRENTOS-M standard interface `if_OS_Storage` by still relying on the `StorageServer`
- we have to replace our existing storage backend (the `RamDisk` component) by the `RPi_SPI_Flash` component (see `/sdk/components/RPi_SPI_Flash`)
- as the `RPi_SPI_Flash` component requires an additional timer source, we have to add support for the `TimeServer` component (see `/sdk/components/TimeServer`)

Regarding the required modifications, please have a look at the requirements documented in the TRENTOS-M Handbook

- chapter "TRENTOS-M RPi_SPI_Flash" and
- chapter "TRENTOS-M TimeServer"

as well as the RPi 3 B+ IoT Demo (TRENTOS-M Handbook, chapter "TRENTOS-M Demo IoT App for RPi3B+").

3 How to start

In order to solve this assignment, you can follow the guide provided below:

1. Wire the RPi 3 B+ and the SPI NOR flash breakout board according to the instructions provided by the Development Kit (or alternatively see TRENTOS-M Handbook, chapter "TRENTOS-M RPi_SPI_Flash")
2. Read and understand the TRENTOS-M Handbook
    a. chapter "TRENTOS-M RPi_SPI_Flash" and
    b. chapter "TRENTOS-M TimeServer"
3. Recap the use of the `RamDisk` and `if_OS_Storage` interface in the current setup
4. For both the `RPi_SPI_Flash` and the `TimeServer` adapt the respective locations in the
    a. `CMakeLists.txt` (declaration of a component in CMake)
    b. `DemoTrainingCourse.camkes` (declaration, instantiation and configuration of a component in CAmkES)
5. The NOR flash memory provides a total of 8 MiB. As formatting our disk may take a while, we adapt the setup for a "virtual" disk size of 64 KiB, splitting both partitions again in halves. Hint: have a look at `system_config.h`.

A successful run should provide you with an output similar like the one below:

```
U-Boot 2020.07-rc3-00002-g4b6f96a0a6 (May 26 2020 - 18:38:13 +0200)

DRAM:  948 MiB
RPI 3 Model B+ (0xa020d3)
MMC:   mmc@7e202000: 0, sdhci@7e300000: 1
Loading Environment from FAT... OK
In:    serial
Out:   vidconsole
Err:   vidconsole
Net:   No ethernet found.
starting USB...
Bus usb@7e980000: USB DWC2
scanning bus usb@7e980000 for devices... 4 USB Device(s) found
       scanning usb for storage devices... 0 Storage Device(s) found
Hit any key to stop autoboot:  0
4897508 bytes read in 206 ms (22.7 MiB/s)
## Starting application at 0x007e4000 ...

ELF-loader started on CPU: ARM Ltd. Cortex-A53 r0p4
  paddr=[7e4000..c73cc7]
No DTB passed in from boot loader.
Looking for DTB in CPIO archive...found at 8f0294.
Loaded DTB from 8f0294.
```

```
    paddr=[3c000..3ffff]
ELF-loading image 'kernel'
  paddr=[0..3bfff]
  vaddr=[e0000000..e003bfff]
  virt_entry=e0000000
ELF-loading image 'capdl-loader'
  paddr=[40000..3e8fff]
  vaddr=[10000..3b8fff]
  virt_entry=1860c
Enabling MMU and paging
Jumping to kernel-image entry point...

Bootstrapping kernel
Booting all finished, dropped to user space
main@main.c:2114 Starting CapDL Loader...
main@main.c:2116 CapDL Loader done, suspending...
spt_handle_irq@spt.c:172 handle irq called when no interrupt pending

   INFO: /host/sdk/components/RPi_SPI_Flash/RPi_SPI_Flash.c:132: BCM2837_SPI_Flash init
   INFO: /host/sdk/components/RPi_SPI_Flash/RPi_SPI_Flash.c:197: BCM2837_SPI_Flash done
   INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:504: storage
medium size: 8388608 bytes
   INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:522: client 1:
offset=0, size=32768
   INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:522: client 2:
offset=32768, size=32768
   INFO: /host/sdk/libs/os_filesystem/src/OS_FileSystem.c:125: Maximizing file system
according to size reported by the storage layer (32768 bytes)
   INFO: /host/sdk/libs/os_filesystem/src/lib/SpifFs.c:182: Using SPIFFS (phys_addr = 0,
phys_size = 32768, phys_erase_block = 4096, log_block_size = 4096, log_page_size = 256)
   INFO: /host/sdk/libs/os_filesystem/src/OS_FileSystem.c:125: Maximizing file system
according to size reported by the storage layer (32768 bytes)
   INFO: /host/sdk/libs/os_filesystem/src/lib/SpifFs.c:182: Using SPIFFS (phys_addr = 0,
phys_size = 32768, phys_erase_block = 4096, log_block_size = 4096, log_page_size = 256)
   INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:135:
Demo completed successfully.
```

Code Block 1: RPi 3 B+ Terminal

HENSOLDT Cyber GmbH

Willy-Messerschmitt-Straße 3
82024 Taufkirchen,
Deutschland

www.hensoldt-cyber.com